

105. Construct Binary Tree from Preorder and Inorder Traversal

Input is a BST

PLATFORM

Platform: LeetCode

DIFFICULTY

MEDIUM

DATE

Solved: August 4, 2026

Problem Summary

Given preorder and inorder traversal arrays of a binary tree, reconstruct and return the original binary tree.

Approach

Preorder's first element is always the root. Find that root in inorder — everything left is the left subtree, everything right is the right subtree. Recursively build both subtrees using correctly computed index ranges.

Algo

1. Build HashMap — $\text{inorder}[i] \rightarrow i$ for $O(1)$ root lookup.
2. $\text{build}(\text{preStart}, \text{preEnd}, \text{inStart}, \text{inEnd})$:
 - If $\text{preStart} > \text{preEnd}$ return null.
 - $\text{rootVal} = \text{pre}[\text{preStart}]$ — first element is root.
 - Find $\text{mid} = \text{map.get}(\text{rootVal})$ — root's index in inorder.
 - $\text{leftSize} = \text{mid} - \text{inStart}$.
 - Recurse left: $\text{build}(\text{preStart}+1, \text{preStart}+\text{leftSize}, \text{inStart}, \text{mid}-1)$.
 - Recurse right: $\text{build}(\text{preStart}+\text{leftSize}+1, \text{preEnd}, \text{mid}+1, \text{inEnd})$.
3. Return root.

Key Insights

- Preorder $\rightarrow$ root always at preStart (first element of current subarray).
- Inorder $\rightarrow$ root splits array into left and right subtrees.
- $\text{leftSize} = \text{mid} - \text{inStart}$ — number of nodes in left subtree, used to split preorder subarray.
- HashMap replaces linear search — reduces $O(n^2)$ to $O(n)$.

Time Complexity & Space Complexity

- Brute force: $O(n^2)$ — linear search at every call.
- Optimal: $O(n)$ — HashMap lookup $O(1)$ per call $\times n$ calls.

$O(n)$ — HashMap + recursion stack $O(h)$.

Approaches

- **Brute Force $O(n^2)$** : Linear search to find root in inorder at every recursive call. $O(n)$ per call $\times O(n)$ calls = $O(n^2)$ time.
- **Optimal $O(n)$** : Precompute a HashMap of value $\rightarrow$ index for inorder array. Root lookup becomes $O(1)$ $\rightarrow$ total $O(n)$ time.

Problem List

Description

Editorial

Solutions

Submissions

105. Construct Binary Tree from Preorder and Inorder Traversal

Solved

Medium

Topics

Companies

Given two integer arrays `preorder` and `inorder` where `preorder` is the preorder traversal of a binary tree and `inorder` is the inorder traversal of the same tree, construct and return the *binary tree*.

Example 1:

```
graph TD
    3((3)) --> 9((9))
    3 --> 20((20))
    9 --> 15_1((15))
    20 --> 15_2((15))
    20 --> 7((7))
```

Input: `preorder = [3,9,20,15,7]`, `inorder = [9,3,15,20,7]`
Output: `[3,9,20,null,null,15,7]`

Example 2:

Input: `preorder = [-1]`, `inorder = [-1]`
Output: `[-1]`

Constraints:

- `1 <= preorder.length <= 3000`
- `inorder.length == preorder.length`
- `-3000 <= preorder[i], inorder[i] <= 3000`
- `preorder` and `inorder` consist of **unique values**.
- Each value of `inorder` also appears in `preorder`.

16.8K 367 155 Online

Accepted

All Submissions

Accepted 203 / 203 testcases passed

Sahil Khan submitted at Aug 04, 2026 22:47

1583.25ms 99.99%

Unlock the Full LeetCode Experience

Company problems, Ask Leet, and expert editor...

Runtime

3 ms Beats 28.48%

Memory

45.35 MB Beats 89.46%

Code

Java

```
1 /**
2  * Definition for a binary tree node.
3  * public class TreeNode {
4  *     int val;
5  *     TreeNode left;
6  *     TreeNode right;
7  *     TreeNode() {}
8  *     TreeNode(int val) { this.val = val; }
9  * }
10 */
11 class Solution {
12     public TreeNode buildTree(int[] preorder, int[] inorder) {
13         return build(preorder, 0, preorder.length-1, inorder, 0, inorder.length-1);
14     }
15
16     private TreeNode build(int[] pre, int preStart, int preEnd, int[] in, int inStart, int inEnd) {
17         if(preStart > preEnd) return null;
18
19         int rootVal = pre[preStart];
20         TreeNode root = new TreeNode(rootVal);
21
22         int mid = 0;
23         for(int i = inStart; i <= inEnd; i++){
24             if(in[i] == rootVal){ mid = i; break; }
25         }
26
27         int leftSize = mid - inStart;
28
29         root.left = build(pre, preStart+1, preStart+leftSize, in, inStart, mid-1);
30
31         root.right = build(pre, preStart+leftSize+1, preEnd, in, mid+1, inEnd);
32         return root;
33     }
34 }
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

preorder =

[3,9,20,15,7]

inorder =

[9,3,15,20,7]

More challenges

106. Construct Binary Tree from Inorder and Postorder Traversal